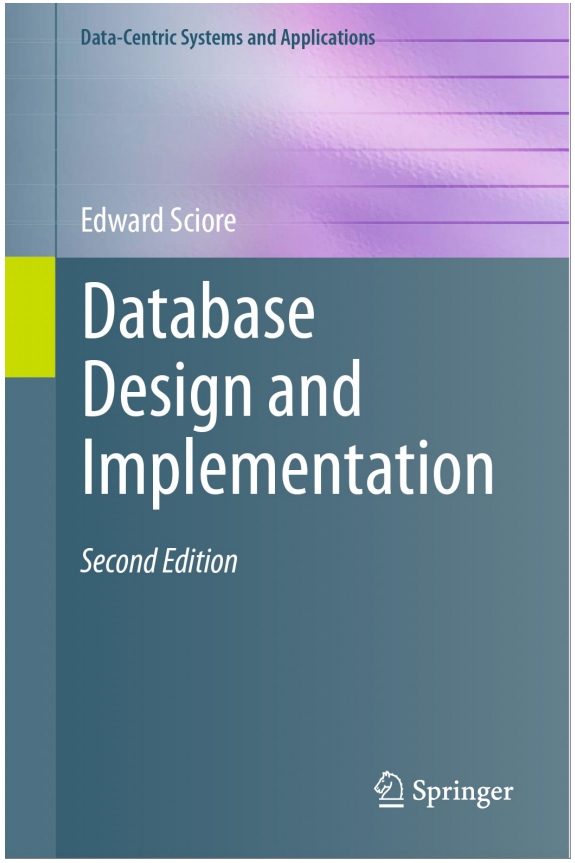
SimpleDB, parte 8

Luigi Bellomarini
luigi.bellomarini@uniroma3.it



SimpleDB, componenti per le transazioni

JDBC: riceve le richieste dal client e passa SQL a Planner

Planner: chiama Parser e determina piano e lo passa al Query

Parser: analisi sintattica

Query: riceve il piano e chiama il Record per ogni tabella

Metadata: gestisce gli schemi delle tabelle

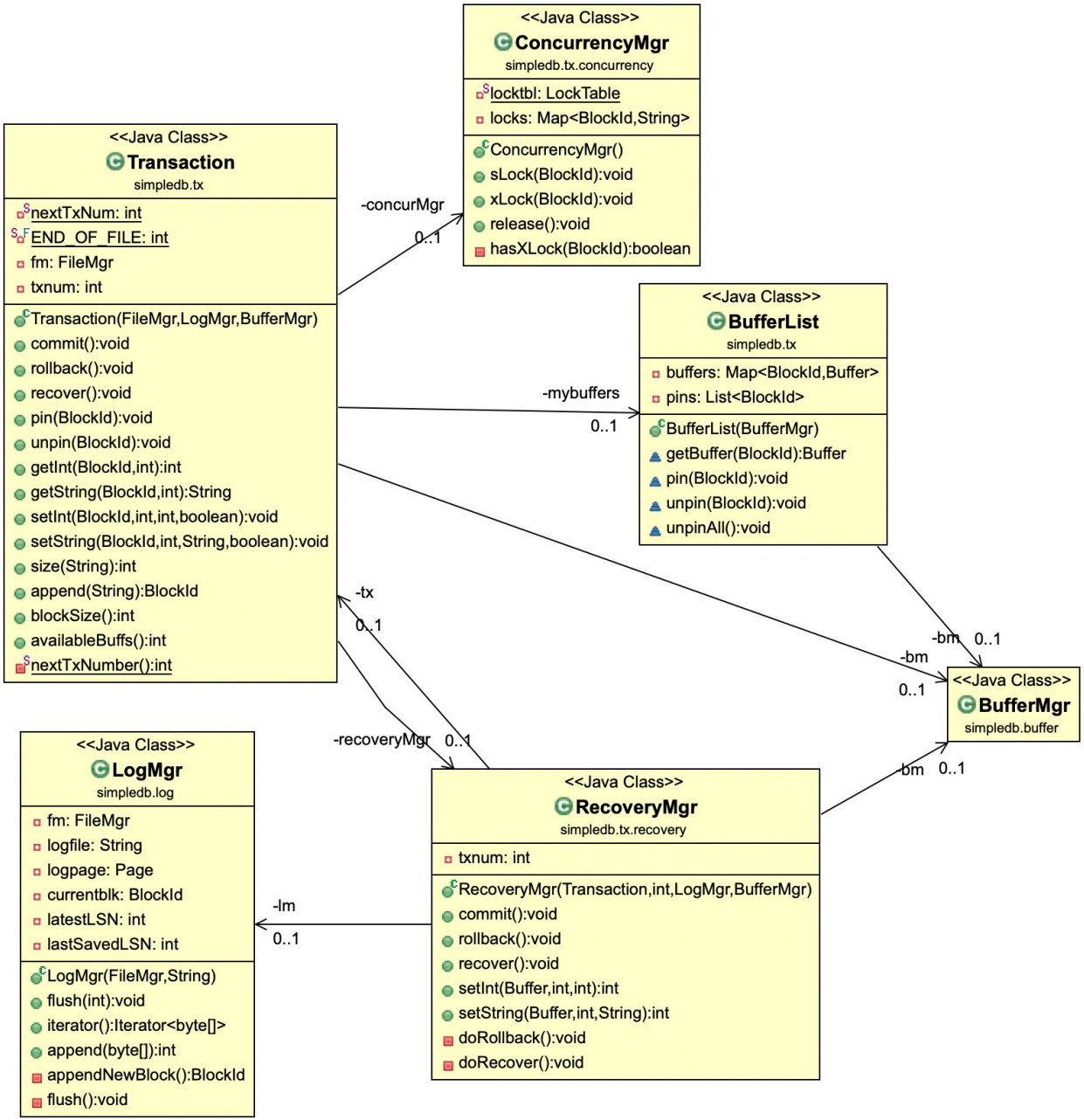
Record: gestisce i blocchi per i record delle tabelle

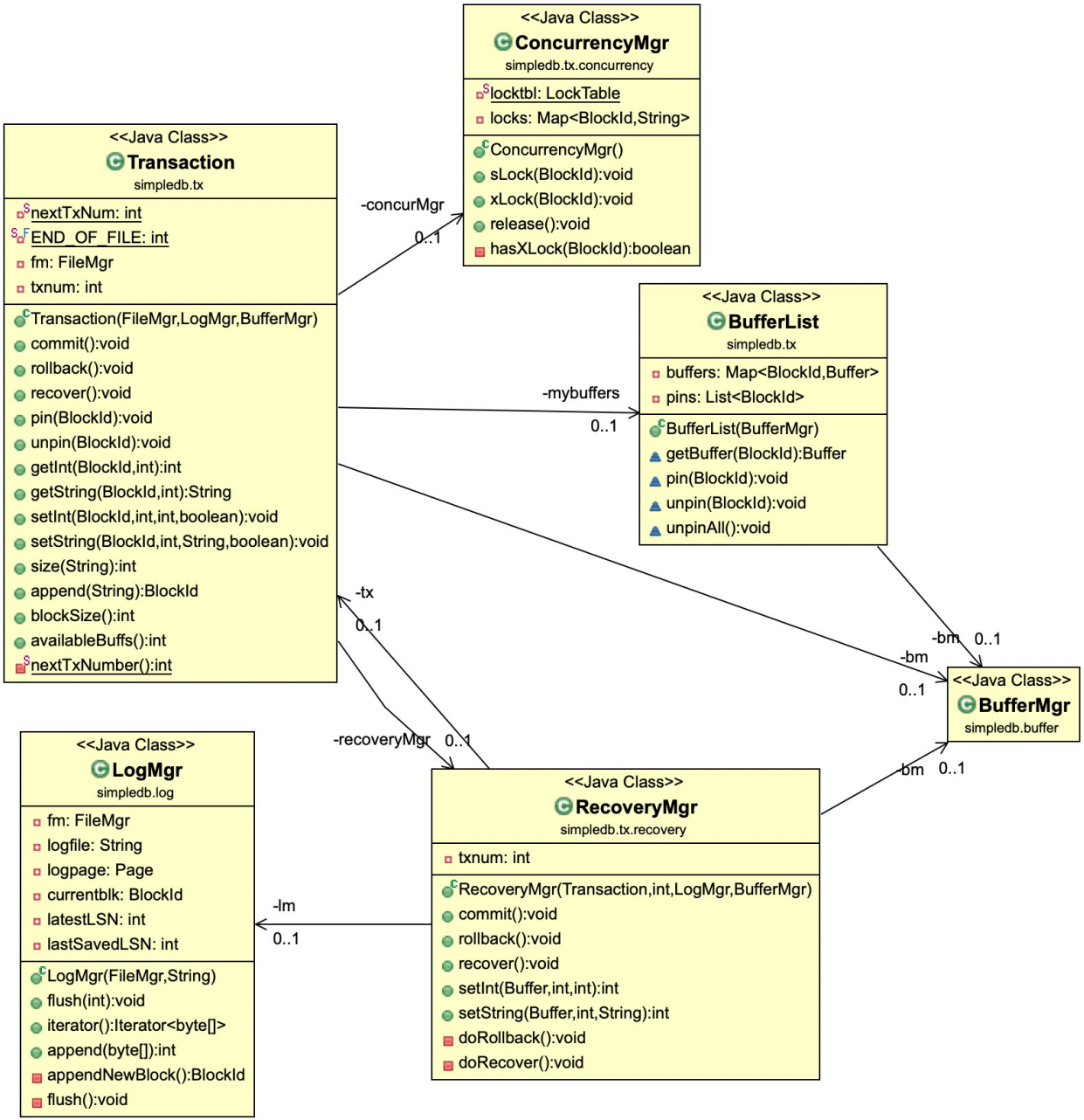
Concurrency Manager: gestisce la concorrenza

Buffer: mantiene in memoria le pagine per limitare gli accessi

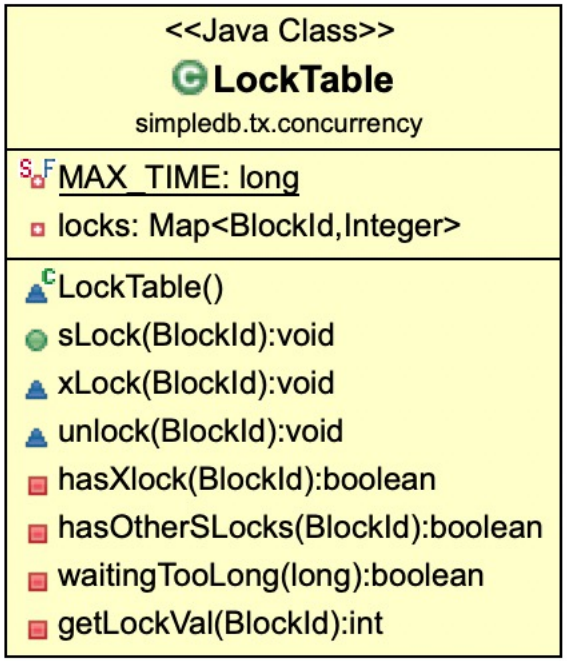
Recovery Manager: garantisce l'affidabilità

File: legge e scrive le pagine su disco

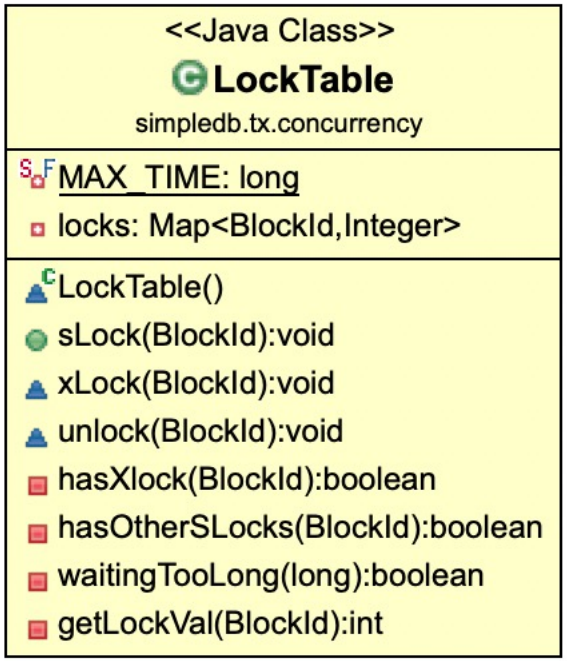

<<Java Class>>
ConcurrencyMgr
simpledb.tx.concurrency
locktbl: LockTable
locks: Map<BlockId,String>
ConcurrencyMgr()
sLock(BlockId):void
xLock(BlockId):void
release():void
hasXLock(BlockId):boolean
<<Java Class>>
Transaction
simpledb.tx
nextTxNum: int
END_OF_FILE: int
fm: FileMgr
txnum: int
Transaction(FileMgr,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
pin(BlockId):void
unpin(BlockId):void
getInt(BlockId,int):int
getString(BlockId,int):String
setInt(BlockId,int,int,boolean):void
setString(BlockId,int,String,boolean):void
size(String):int
append(String):BlockId
blockSize():int
availableBuffs():int
nextTxNumber():int
-concurMgr
0..1
<<Java Class>>
BufferList
simpledb.tx
buffers: Map<BlockId,Buffer>
pins: List<BlockId>
BufferList(BufferMgr)
getBuffer(BlockId):Buffer
pin(BlockId):void
unpin(BlockId):void
unpinAll():void
-mybuffers
0..1
-tx
0..1
-bm
0..1
-bm
0..1
<<Java Class>>
BufferMgr
simpledb.buffer
-recoveryMgr
0..1
-bm
0..1
<<Java Class>>
LogMgr
simpledb.log
fm: FileMgr
logfile: String
logpage: Page
currentblk: BlockId
latestLSN: int
lastSavedLSN: int
LogMgr(FileMgr,String)
flush(int):void
iterator():Iterator<byte[]>
append(byte[]):int
appendNewBlock():BlockId
flush():void
-lm
0..1
<<Java Class>>
RecoveryMgr
simpledb.tx.recovery
txnum: int
RecoveryMgr(Transaction,int,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
setInt(Buffer,int,int):int
setString(Buffer,int,String):int
doRollback():void
doRecover():void

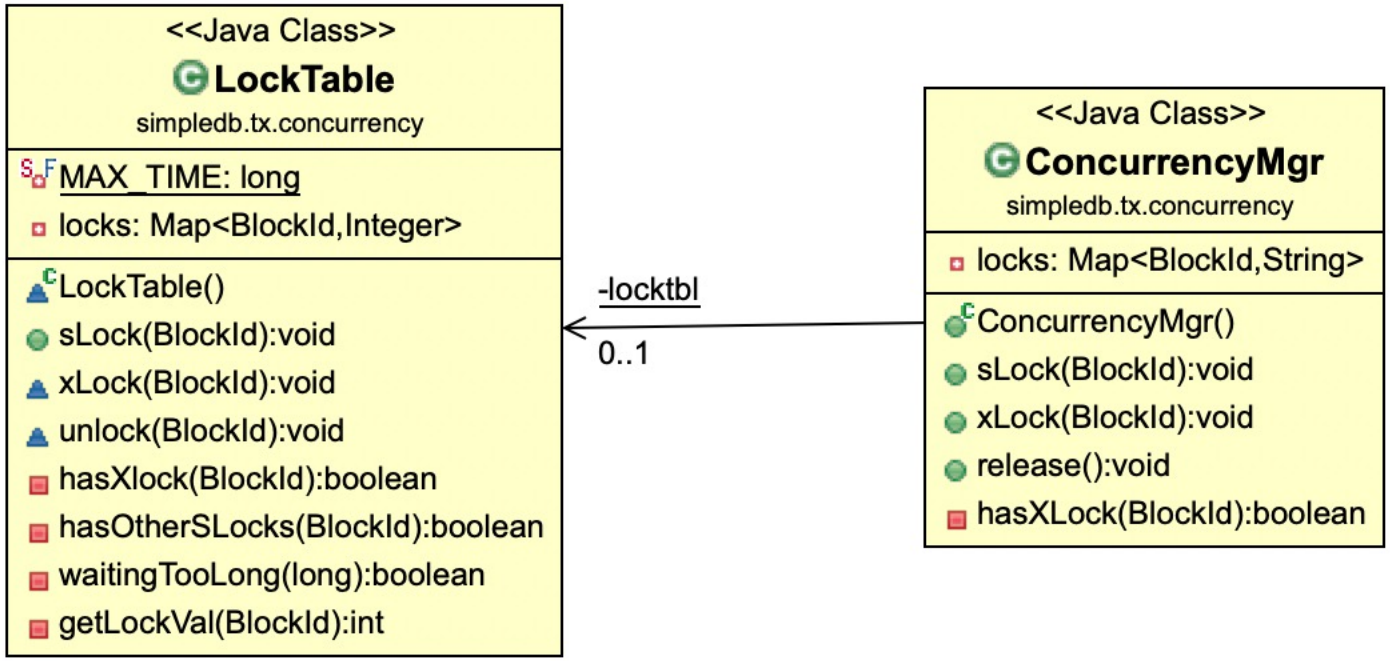

<<Java Class>>
ConcurrencyMgr
simpledb.tx.concurrency
locktbl: LockTable
locks: Map<BlockId,String>
ConcurrencyMgr()
sLock(BlockId):void
xLock(BlockId):void
release():void
hasXLock(BlockId):boolean
<<Java Class>>
Transaction
simpledb.tx
nextTxNum: int
END_OF_FILE: int
fm: FileMgr
txnum: int
Transaction(FileMgr,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
pin(BlockId):void
unpin(BlockId):void
getInt(BlockId,int):int
getString(BlockId,int):String
setInt(BlockId,int,int,boolean):void
setString(BlockId,int,String,boolean):void
size(String):int
append(String):BlockId
blockSize():int
availableBuffs():int
nextTxNumber():int
-concurMgr
0..1
<<Java Class>>
BufferList
simpledb.tx
buffers: Map<BlockId,Buffer>
pins: List<BlockId>
BufferList(BufferMgr)
getBuffer(BlockId):Buffer
pin(BlockId):void
unpin(BlockId):void
unpinAll():void
-mybuffers
0..1
-tx
0..1
-bm
0..1
-bm
0..1
<<Java Class>>
BufferMgr
simpledb.buffer
-recoveryMgr
0..1
-bm
0..1
<<Java Class>>
LogMgr
simpledb.log
fm: FileMgr
logfile: String
logpage: Page
currentblk: BlockId
latestLSN: int
lastSavedLSN: int
LogMgr(FileMgr,String)
flush(int):void
iterator():Iterator<byte[]>
append(byte[]):int
appendNewBlock():BlockId
flush():void
<<Java Class>>
RecoveryMgr
simpledb.tx.recovery
txnum: int
RecoveryMgr(Transaction,int,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
setInt(Buffer,int,int):int
setString(Buffer,int,String):int
doRollback():void
doRecover():void
-lm
0..1

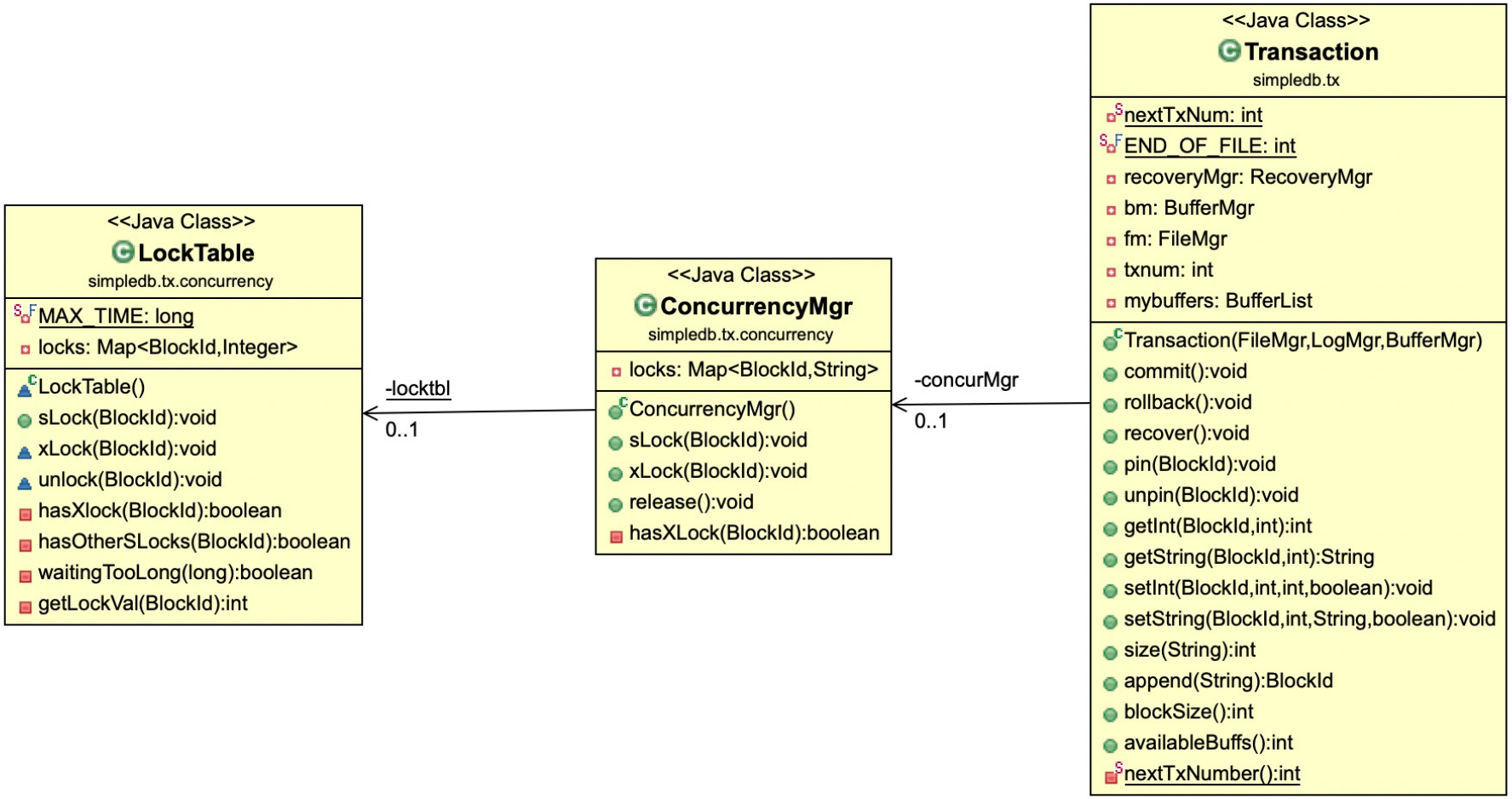
Lock Table

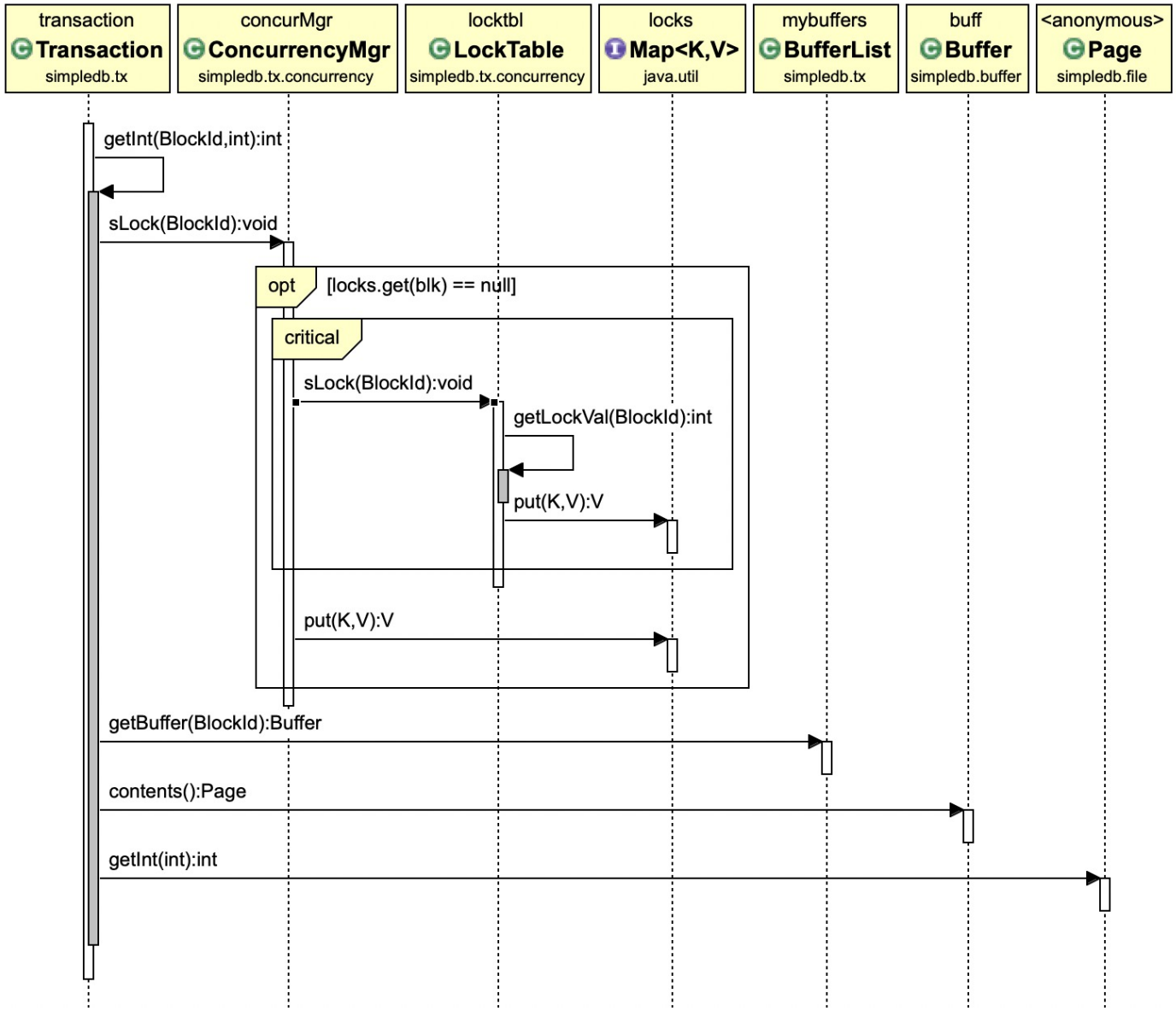

<<Java Class>>
LockTable
simpledb.tx.concurrency
MAX_TIME: long
locks: Map<BlockId,Integer>
LockTable()
sLock(BlockId):void
xLock(BlockId):void
unlock(BlockId):void
hasXlock(BlockId):boolean
hasOtherSLocks(BlockId):boolean
waitingTooLong(long):boolean
getLockVal(BlockId):int

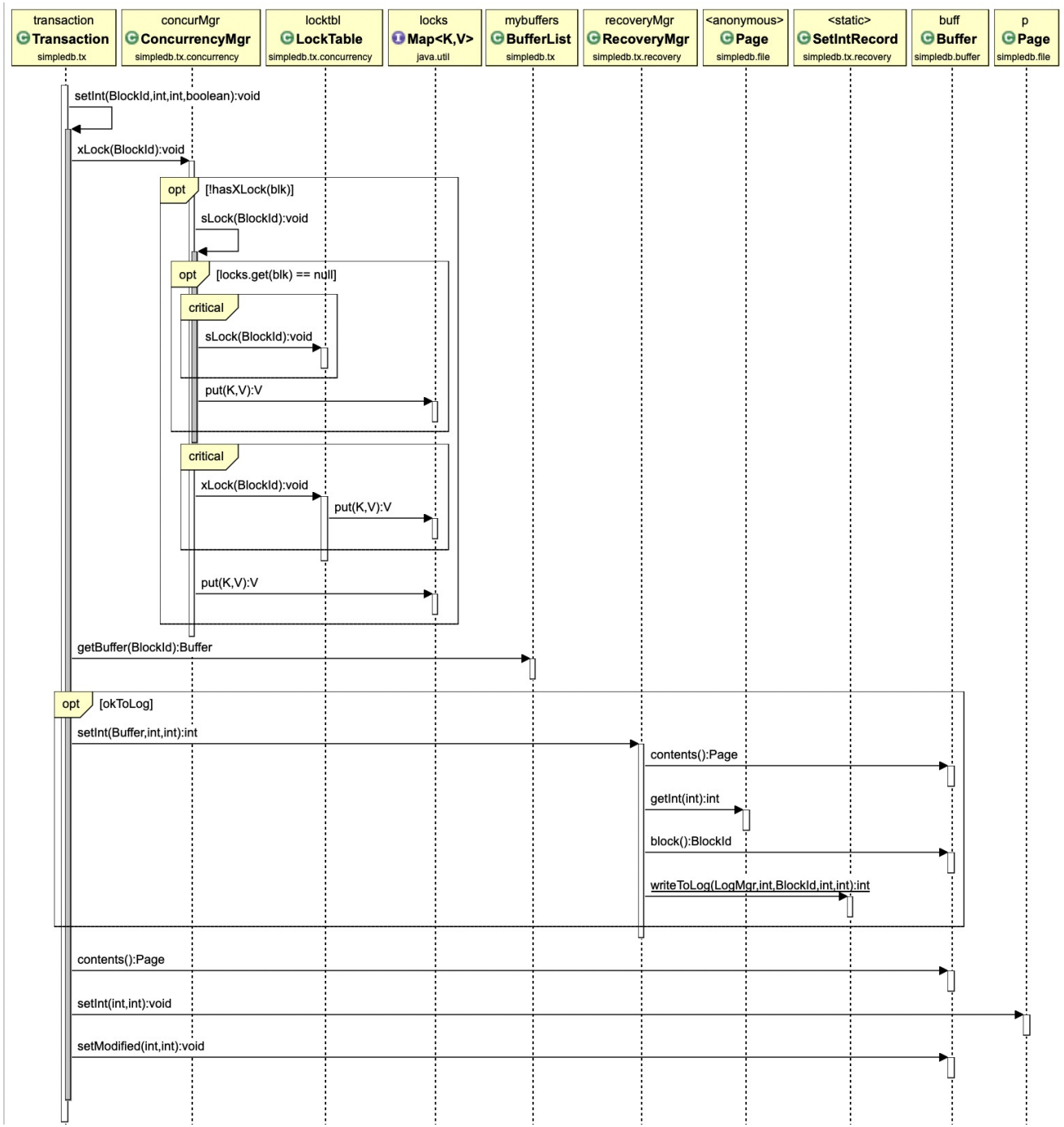
Lock Table

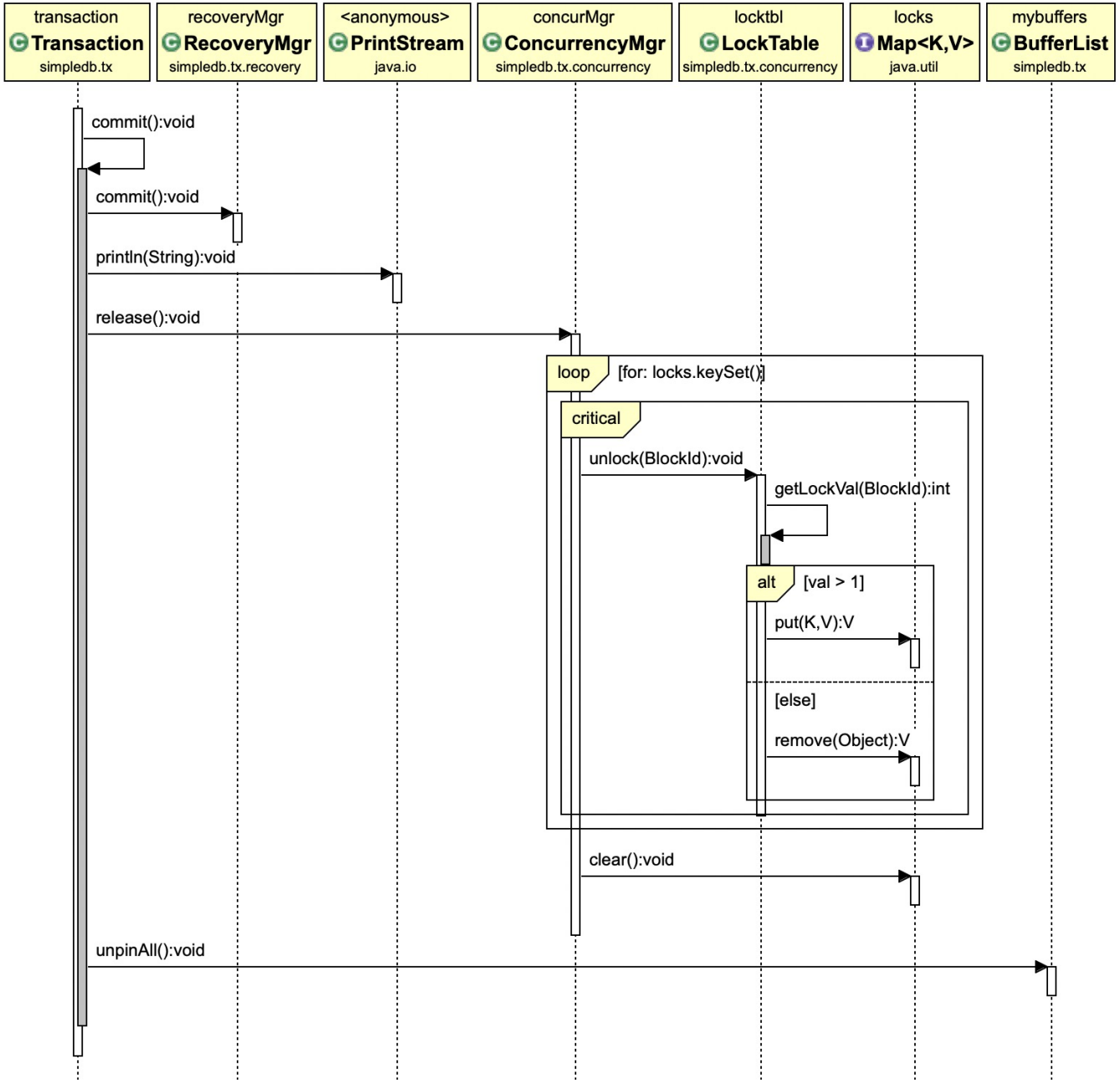

<<Java Class>>
LockTable
simpledb.tx.concurrency
MAX_TIME: long
locks: Map<BlockId,Integer>
LockTable()
sLock(BlockId):void
xLock(BlockId):void
unlock(BlockId):void
hasXlock(BlockId):boolean
hasOtherSLocks(BlockId):boolean
waitingTooLong(long):boolean
getLockVal(BlockId):int


<<Java Class>>
LockTable
simpledb.tx.concurrency
MAX_TIME: long
locks: Map<BlockId,Integer>
LockTable()
sLock(BlockId):void
xLock(BlockId):void
unlock(BlockId):void
hasXlock(BlockId):boolean
hasOtherSLocks(BlockId):boolean
waitingTooLong(long):boolean
getLockVal(BlockId):int
-locktbl
0..1
<<Java Class>>
ConcurrencyMgr
simpledb.tx.concurrency
locks: Map<BlockId,String>
ConcurrencyMgr()
sLock(BlockId):void
xLock(BlockId):void
release():void
hasXLock(BlockId):boolean


<<Java Class>>
LockTable
simpledb.tx.concurrency
MAX_TIME: long
locks: Map<BlockId,Integer>
LockTable()
sLock(BlockId):void
xLock(BlockId):void
unlock(BlockId):void
hasXlock(BlockId):boolean
hasOtherSLocks(BlockId):boolean
waitingTooLong(long):boolean
getLockVal(BlockId):int
-locktbl
0..1
<<Java Class>>
ConcurrencyMgr
simpledb.tx.concurrency
locks: Map<BlockId,String>
ConcurrencyMgr()
sLock(BlockId):void
xLock(BlockId):void
release():void
hasXLock(BlockId):boolean
-concurMgr
0..1
<<Java Class>>
Transaction
simpledb.tx
nextTxNum: int
END_OF_FILE: int
recoveryMgr: RecoveryMgr
bm: BufferMgr
fm: FileMgr
txnum: int
mybuffers: BufferList
Transaction(FileMgr,LogMgr,BufferMgr)
commit():void
rollback():void
recover():void
pin(BlockId):void
unpin(BlockId):void
getInt(BlockId,int):int
getString(BlockId,int):String
setInt(BlockId,int,int,boolean):void
setString(BlockId,int,String,boolean):void
size(String):int
append(String):BlockId
blockSize():int
availableBuffs():int
nextTxNumber():int


transaction
Transaction
simpledb.tx
concurMgr
ConcurrencyMgr
simpledb.tx.concurrency
locktbl
LockTable
simpledb.tx.concurrency
locks
Map<K,V>
java.util
mybuffers
BufferList
simpledb.tx
buff
Buffer
simpledb.buffer
<anonymous>
Page
simpledb.file
getInt(BlockId,int):int
sLock(BlockId):void
opt
[locks.get(blk) == null]
critical
sLock(BlockId):void
getLockVal(BlockId):int
put(K,V):V
put(K,V):V
getBuffer(BlockId):Buffer
contents():Page
getInt(int):int


transaction
Transaction
simpledb.tx
concurMgr
ConcurrencyMgr
simpledb.tx.concurrency
locktbl
LockTable
simpledb.tx.concurrency
locks
Map<K,V>
java.util
mybuffers
BufferList
simpledb.tx
recoveryMgr
RecoveryMgr
simpledb.tx.recovery
<anonymous>
Page
simpledb.file
<static>
SetIntRecord
simpledb.tx.recovery
buff
Buffer
simpledb.buffer
p
Page
simpledb.file
setInt(BlockId,int,int,boolean):void
xLock(BlockId):void
opt
[!hasXLock(blk)]
sLock(BlockId):void
opt
[locks.get(blk) == null]
critical
sLock(BlockId):void
put(K,V):V
critical
xLock(BlockId):void
put(K,V):V
put(K,V):V
getBuffer(BlockId):Buffer
opt
[okToLog]
setInt(Buffer,int,int):int
contents():Page
getInt(int):int
block():BlockId
writeToLog(LogMgr,int,BlockId,int,int):int
contents():Page
setInt(int,int):void
setModified(int,int):void


transaction
Transaction
simpledb.tx
recoveryMgr
RecoveryMgr
simpledb.tx.recovery
<anonymous>
PrintStream
java.io
concurMgr
ConcurrencyMgr
simpledb.tx.concurrency
locktbl
LockTable
simpledb.tx.concurrency
locks
Map<K,V>
java.util
mybuffers
BufferList
simpledb.tx
commit():void
commit():void
println(String):void
release():void
loop
[for: locks.keySet()]
critical
unlock(BlockId):void
getLockVal(BlockId):int
alt
[val > 1]
put(K,V):V
[else]
remove(Object):V
clear():void
unpinAll():void

Esercizio (approfondimento)

- Abbiamo visto che una transazione di SimpleDB acquisisce i lock sui blocchi quando vengono invocati i metodi getInt o getString.

- Un'altra possibilità è invece che una transazione acquisisca il lock condiviso, quando viene effettuata la pin di un blocco.
  - L'assunzione è che una pin è effettuata solo quando si vuole almeno leggere il contenuto di un blocco.

- Implementare questa strategia in SimpleDB e discuterne vantaggi/svantaggi.